R1-Zero and Pure RL Reasoning

Deep Dive into DeepSeek — Chapter 8

Yin Yang

Foundation Books Project

The one rule of this chapter

A rule cannot optimize properties that it never measures.

- DeepSeek-R1-Zero applies large-scale reinforcement learning directly to a base model, DeepSeek-V3-Base, with no supervised cold-start stage first.
- A base policy samples completions; an extractor isolates a gradable answer; a rule-based verifier turns that answer into reward.
- Within-prompt comparisons yield group-relative advantages, and a guarded update moves the policy toward what the group rewarded.

The one rule of this chapter

A rule cannot optimize properties that it never measures.

- DeepSeek-R1-Zero applies large-scale reinforcement learning directly to a base model, DeepSeek-V3-Base, with no supervised cold-start stage first.
- A base policy samples completions; an extractor isolates a gradable answer; a rule-based verifier turns that answer into reward.
- Within-prompt comparisons yield group-relative advantages, and a guarded update moves the policy toward what the group rewarded.

Goal: follow one small congruence example through reward, group comparison, and update pressure, then read the reported R1-Zero behaviors — and their limits — against that mechanism.

Roadmap

1. From a base policy to a rule-verifiable reward

Roadmap

1. From a base policy to a rule-verifiable reward
2. GRPO: group-relative comparison

Roadmap

1. From a base policy to a rule-verifiable reward
2. GRPO: group-relative comparison
3. The objective, KL placement, and token tensors

Roadmap

1. From a base policy to a rule-verifiable reward
2. GRPO: group-relative comparison
3. The objective, KL placement, and token tensors
4. What emerged, and what broke

Roadmap

1. From a base policy to a rule-verifiable reward
2. GRPO: group-relative comparison
3. The objective, KL placement, and token tensors
4. What emerged, and what broke
5. Fit, evidence, and what carries forward

From a base policy to a rule-verifiable reward

The running example: one prompt, four rollouts

Find $n \in \{0, \ldots, 34\}$ such that $n \bmod 5 = 2$ and $n \bmod 7 = 3$. Unique answer: 17.

base policy
$\pi_0(o \mid q)$

- A base policy $\pi_0(o \mid q)$ samples completions o for prompt q. An extractor E maps a completion to a gradable answer or to $\emptyset$. A verifier $V(q, E(o))$ returns $\{0, 1\}$.
- The verifier scores only the extracted object — not the full completion, and not an intuition about whether the prose looks plausible.

The running example: one prompt, four rollouts

Find $n \in \{0, \dots, 34\}$ such that $n \bmod 5 = 2$ and $n \bmod 7 = 3$. Unique answer: 17.

$$\begin{tikzcd} \boxed{\text{base policy } \pi_0(o \mid q)} \arrow[r, "\text{samples}"] & \boxed{\text{completions } o} \end{tikzcd}$$

- A base policy $\pi_0(o \mid q)$ samples completions o for prompt q. An extractor E maps a completion to a gradable answer or to $\emptyset$. A verifier $V(q, E(o))$ returns $\{0, 1\}$.
- The verifier scores only the extracted object — not the full completion, and not an intuition about whether the prose looks plausible.

The running example: one prompt, four rollouts

Find $n \in \{0, \ldots, 34\}$ such that $n \bmod 5 = 2$ and $n \bmod 7 = 3$. Unique answer: 17.

- A base policy $\pi_0(o \mid q)$ samples completions o for prompt q. An extractor E maps a completion to a gradable answer or to $\emptyset$. A verifier $V(q, E(o))$ returns $\{0, 1\}$.
- The verifier scores only the extracted object — not the full completion, and not an intuition about whether the prose looks plausible.

The running example: one prompt, four rollouts

Find $n \in \{0, \ldots, 34\}$ such that $n \bmod 5 = 2$ and $n \bmod 7 = 3$. Unique answer: 17.

- A base policy $\pi_0(o \mid q)$ samples completions o for prompt q. An extractor E maps a completion to a gradable answer or to $\emptyset$. A verifier $V(q, E(o))$ returns $\{0, 1\}$.
- The verifier scores only the extracted object — not the full completion, and not an intuition about whether the prose looks plausible.

The running example: one prompt, four rollouts

Find $n \in \{0, \ldots, 34\}$ such that $n \bmod 5 = 2$ and $n \bmod 7 = 3$. Unique answer: 17.

- A base policy $\pi_0(o \mid q)$ samples completions o for prompt q. An extractor E maps a completion to a gradable answer or to $\emptyset$. A verifier $V(q, E(o))$ returns $\{0, 1\}$.
- The verifier scores only the extracted object — not the full completion, and not an intuition about whether the prose looks plausible.

This asymmetry — cheap to check the answer, hard to check the reasoning — is the seed of both this chapter's success story and its failure modes.

Accuracy, format, and total reward stay separate fields

Candidate	Extracted answer	$r_{\mathrm{acc}},\ r_{\mathrm{fmt}}$	r
o_1	17, tagged	1, 1	1.1

$r(q, o) = r_{\mathrm{acc}}(q, o) + \lambda\, r_{\mathrm{fmt}}(o)$, toy weight $\lambda = 0.1$.

Accuracy, format, and total reward stay separate fields

Candidate	Extracted answer	$r_{\mathrm{acc}},\ r_{\mathrm{fmt}}$	r
o_1	17, tagged	1, 1	1.1
o_2	32, tagged	0, 1	0.1

$r(q, o) = r_{\mathrm{acc}}(q, o) + \lambda\, r_{\mathrm{fmt}}(o)$, toy weight $\lambda = 0.1$.

Accuracy, format, and total reward stay separate fields

Candidate	Extracted answer	$r_{\mathrm{acc}},\ r_{\mathrm{fmt}}$	r
o_1	17, tagged	1, 1	1.1
o_2	32, tagged	0, 1	0.1
o_3	untagged prose	0, 0	0.0

$r(q, o) = r_{\mathrm{acc}}(q, o) + \lambda\, r_{\mathrm{fmt}}(o)$, toy weight $\lambda = 0.1$.

Accuracy, format, and total reward stay separate fields

Candidate	Extracted answer	$r_{\mathrm{acc}},\ r_{\mathrm{fmt}}$	r
o_1	17, tagged	1, 1	1.1
o_2	32, tagged	0, 1	0.1
o_3	untagged prose	0, 0	0.0
o_4	17, untagged	1, 0	1.0

$r(q,o) = r_{\mathrm{acc}}(q,o) + \lambda\, r_{\mathrm{fmt}}(o)$, toy weight $\lambda = 0.1$.

Accuracy, format, and total reward stay separate fields

Candidate	Extracted answer	$r_{\mathrm{acc}},\ r_{\mathrm{fmt}}$	r
o_1	17, tagged	1, 1	1.1
o_2	32, tagged	0, 1	0.1
o_3	untagged prose	0, 0	0.0
o_4	17, untagged	1, 0	1.0

$r(q,o) = r_{\mathrm{acc}}(q,o) + \lambda\, r_{\mathrm{fmt}}(o)$, toy weight $\lambda = 0.1$.

Correctness and formatting are different objects. Neither implies the other — o_2 is formatted but wrong, o_4 is right but unformatted.

GRPO: group-relative comparison

The group supplies its own baseline

Definition (Group-relative advantage)
For a prompt q, sample G completions from $\pi_{\theta_{\mathrm{old}}}$. With rewards $r_1, \ldots, r_G$ and $\sigma_r \neq 0$,

$$A_i = \frac{r_i - \mu_r}{\sigma_r}, \qquad \mu_r = \frac{1}{G}\sum_j r_j.$$

When $\sigma_r = 0$, set $A_i = 0$ for every i.

The group supplies its own baseline

Definition (Group-relative advantage)
For a prompt q, sample G completions from $\pi_{\theta_{\text{old}}}$. With rewards $r_1, \ldots, r_G$ and $\sigma_r \neq 0$,

$$A_i = \frac{r_i - \mu_r}{\sigma_r}, \qquad \mu_r = \frac{1}{G}\sum_j r_j.$$

When $\sigma_r = 0$, set $A_i = 0$ for every i.

For rewards $(1.1, 0.1, 0.0, 1.0)$: $\mu_r = 0.55$, $\sigma_r \approx 0.5025$.

The group supplies its own baseline

Definition (Group-relative advantage)
For a prompt q, sample G completions from $\pi_{\theta_{\text{old}}}$. With rewards $r_1, \ldots, r_G$ and $\sigma_r \neq 0$,

$$A_i = \frac{r_i - \mu_r}{\sigma_r}, \qquad \mu_r = \frac{1}{G} \sum_j r_j.$$

When $\sigma_r = 0$, set $A_i = 0$ for every i.

For rewards $(1.1, 0.1, 0.0, 1.0)$: $\mu_r = 0.55$, $\sigma_r \approx 0.5025$. So $A \approx (1.09, -0.90, -1.09, 0.90)$.

The group supplies its own baseline

Definition (Group-relative advantage)
For a prompt q, sample G completions from $\pi_{\theta_{\text{old}}}$. With rewards $r_1, \ldots, r_G$ and $\sigma_r \neq 0$,

$$A_i = \frac{r_i - \mu_r}{\sigma_r}, \qquad \mu_r = \frac{1}{G}\sum_j r_j.$$

When $\sigma_r = 0$, set $A_i = 0$ for every i.

For rewards $(1.1, 0.1, 0.0, 1.0)$: $\mu_r = 0.55$, $\sigma_r \approx 0.5025$. So $A \approx (1.09, -0.90, -1.09, 0.90)$.

No trained critic supplies this baseline — the sampled group does. DeepSeekMath motivates GRPO by removing the value model; the group still costs G rollouts and G reward evaluations per prompt.

The PPO-to-GRPO bridge

	Actor–critic PPO	GRPO
Update direction	Learned critic, often with GAE	Within-prompt group-relative reward comparison

The PPO-to-GRPO bridge

	Actor–critic PPO	GRPO
Update direction	Learned critic, often with GAE	Within-prompt group-relative reward comparison
Movement guard	Clipped current-to-old ratio	Same PPO-shaped clipped ratio, retained

The PPO-to-GRPO bridge

	Actor–critic PPO	GRPO
Update direction	Learned critic, often with GAE	Within-prompt group-relative reward comparison
Movement guard	Clipped current-to-old ratio	Same PPO-shaped clipped ratio, retained
Reference drift	Optional separate penalty	KL to π_{ref}, subtracted after A_i is formed

The PPO-to-GRPO bridge

	Actor–critic PPO	GRPO
Update direction	Learned critic, often with GAE	Within-prompt group-relative reward comparison
Movement guard	Clipped current-to-old ratio	Same PPO-shaped clipped ratio, retained
Reference drift	Optional separate penalty	KL to π_{ref}, subtracted after A_i is formed

GRPO trades a trained critic for rollout cost. It keeps the clipped-ratio guard exactly where PPO put it.

The objective, KL placement, and token tensors

One objective, three separable jobs

$$\ell_i^{\text{seq}}(\theta) = \mathsf{min}\Big(\rho_i^{\text{seq}} A_i,\ \mathsf{clip}(\rho_i^{\text{seq}}, 1-\epsilon, 1+\epsilon) A_i\Big) - \beta D_{\text{KL},i}^{\text{seq}}$$

- $\rho_i^{\text{seq}} = \pi_\theta(o_i \mid q)/\pi_{\theta_{\text{old}}}(o_i \mid q)$ measures movement; A_i supplies direction and strength.
- Task rewards form A_i first. The KL term to π_{ref} is subtracted after, directly in the loss — it never changes the group's reward mean, spread, or advantage signs.

One objective, three separable jobs

$$\ell_i^{\mathrm{seq}}(\theta) = \min\Big(\rho_i^{\mathrm{seq}} A_i,\ \mathrm{clip}(\rho_i^{\mathrm{seq}}, 1-\epsilon, 1+\epsilon) A_i\Big) - \beta D_{\mathrm{KL},i}^{\mathrm{seq}}$$

- $\rho_i^{\mathrm{seq}} = \pi_\theta(o_i \mid q) / \pi_{\theta_{\mathrm{old}}}(o_i \mid q)$ measures movement; A_i supplies direction and strength.
- Task rewards form A_i first. The KL term to π_{ref} is subtracted after, directly in the loss — it never changes the group's reward mean, spread, or advantage signs.

A reproducible record must say *actor-loss KL* or *in-reward KL*: public trainers use either, and only one matches this paper-level placement. [REPORTED: DeepSeek-R1 paper]

Token tensors: same terms, different weight

Rewards/advantages: shape $[G]$. Token ids, ratios, mask: shape $[G, T]$.

- Response-token loss: $\ell_i^{\mathrm{tok}} = \frac{1}{M_i} \sum_t m_{i,t}[\cdots]$ — variable per-response weight $1/M_i$.
- Dr. GRPO's fixed-divisor variant instead uses $1/T_r$, a shared budget — and drops the σ_r standardization in A_i.
- Equal-response-weight and equal-valid-token-weight reductions of the *same* masked terms gave $J_{\mathrm{response}} \approx 0.024$ versus $J_{\mathrm{valid\ token}} \approx -0.107$ in the chapter's worked trace.

Token tensors: same terms, different weight

Rewards/advantages: shape $[G]$. Token ids, ratios, mask: shape $[G, T]$.

- Response-token loss: $\ell_i^{\text{tok}} = \frac{1}{M_i} \sum_t m_{i,t}[\cdots]$ — variable per-response weight $1/M_i$.
- Dr. GRPO's fixed-divisor variant instead uses $1/T_r$, a shared budget — and drops the σ_r standardization in A_i.
- Equal-response-weight and equal-valid-token-weight reductions of the *same* masked terms gave $J_{\text{response}} \approx 0.024$ versus $J_{\text{valid token}} \approx -0.107$ in the chapter's worked trace.

A reported GRPO loss number is uninterpretable until its reduction is named. [DERIVED: worked token trace, this chapter]

What emerged, and what broke

Reward pressure correlates with reasoning-shaped behavior

- Reported: accuracy improves through training; average response length increases on the training set.
- Reported behaviors: self-verification, reflection, longer chains of thought, exploring alternative approaches — including a notable “aha moment” of mid-solution revisiting.
- AIME 2024 pass@1: 15.6% → 71.0%; majority voting reaches 86.7%. [REPORTED: DeepSeek-R1 paper]

Reward pressure correlates with reasoning-shaped behavior

- Reported: accuracy improves through training; average response length increases on the training set.
- Reported behaviors: self-verification, reflection, longer chains of thought, exploring alternative approaches — including a notable "aha moment" of mid-solution revisiting.
- AIME 2024 pass@1: 15.6% → 71.0%; majority voting reaches 86.7%. [REPORTED: DeepSeek-R1 paper]

These are operational, observational claims about sampled outputs under the reported setup — not evidence about model intent, and not proof that every long trace is useful.

The reward's blind spots survive the pressure

Failure mode	Why the reward allows it	R1 bridge
Poor readability	Prose quality is outside final-answer verification	Cold-start readable traces

The reward's blind spots survive the pressure

Failure mode	Why the reward allows it	R1 bridge
Poor readability	Prose quality is outside final-answer verification	Cold-start readable traces
Language mixing	Verifier may ignore language consistency	Language-consistency reward

The reward's blind spots survive the pressure

Failure mode	Why the reward allows it	R1 bridge
Poor readability	Prose quality is outside final-answer verification	Cold-start readable traces
Language mixing	Verifier may ignore language consistency	Language-consistency reward
Repetition, overlong output	Length needs its own rule or control	Length/readability constraints

The reward's blind spots survive the pressure

Failure mode	Why the reward allows it	R1 bridge
Poor readability	Prose quality is outside final-answer verification	Cold-start readable traces
Language mixing	Verifier may ignore language consistency	Language-consistency reward
Repetition, overlong output	Length needs its own rule or control	Length/readability constraints

R1-Zero has reported drawbacks: poor readability and language mixing. The repository README separately warns about repetition and incoherent output in usage guidance.

[REPORTED: DeepSeek-R1 paper and README]

Fit, evidence, and what carries forward

When pure rule RL is the right tool

- Prefer it when outcomes are reliably verifiable, the reward has been tested for shortcuts, and the base policy already produces enough successes for group contrast.
- Readability, language consistency, and robustness to hidden variants stay outside the reward — they need cold-start data, filters, preference signals, or a stronger teacher.
- A rule-verifiable final artifact (an integer, a passing test suite) does not certify everything about the surrounding behavior.

When pure rule RL is the right tool

- Prefer it when outcomes are reliably verifiable, the reward has been tested for shortcuts, and the base policy already produces enough successes for group contrast.
- Readability, language consistency, and robustness to hidden variants stay outside the reward — they need cold-start data, filters, preference signals, or a stronger teacher.
- A rule-verifiable final artifact (an integer, a passing test suite) does not certify everything about the surrounding behavior.

DeepSeek-R1 adds exactly what pure rule RL cannot supply on its own: cold-start data, rejection-sampled SFT, and staged RL — Chapter 9.

The chapter's own arithmetic, reproduced locally

- `code/chapter_r1_zero_pure_rl_reasoning/toy_reward_and_grpo.py` prints `mean=0.55`, `population_std=0.502494`, and `pass_at_one=0.6875`.
- Those are exactly this chapter's worked mean, standard deviation, and the 16-sample congruence pass@1 check — recomputed, not asserted.

The chapter's own arithmetic, reproduced locally

- `code/chapter_r1_zero_pure_rl_reasoning/toy_reward_and_grpo.py` prints `mean=0.55`, `population_std=0.502494`, and `pass_at_one=0.6875`.
- Those are exactly this chapter's worked mean, standard deviation, and the 16-sample congruence pass@1 check — recomputed, not asserted.

This is the book's own companion harness, not a DeepSeek repository excerpt: it lets every claimed number in the worked example be rerun rather than trusted. [DERIVED: this chapter's companion harness]

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** $\rightarrow$ `DeepSeek-R1/README.md`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** → `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** → `DeepSeek-R1/DeepSeek_R1.pdf`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** → `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** → `DeepSeek-R1/DeepSeek_R1.pdf`
- **group-relative advantage, GRPO origin** → `arXiv:2402.03300 (DeepSeekMath)`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** $\rightarrow$ `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** $\rightarrow$ `DeepSeek-R1/DeepSeek_R1.pdf`
- **group-relative advantage, GRPO origin** $\rightarrow$ `arXiv:2402.03300 (DeepSeekMath)`
- **PPO clipped-ratio ancestor** $\rightarrow$ `arXiv:1707.06347`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** → `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** → `DeepSeek-R1/DeepSeek_R1.pdf`
- **group-relative advantage, GRPO origin** → `arXiv:2402.03300 (DeepSeekMath)`
- **PPO clipped-ratio ancestor** → `arXiv:1707.06347`
- **token-mean loss,** `loss_agg_mode` → `verl.readthedocs.io /en/latest/algo/grpo.html`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** → `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** → `DeepSeek-R1/DeepSeek_R1.pdf`
- **group-relative advantage, GRPO origin** → `arXiv:2402.03300 (DeepSeekMath)`
- **PPO clipped-ratio ancestor** → `arXiv:1707.06347`
- **token-mean loss,** `loss_agg_mode` → `verl.readthedocs.io /en/latest/algo/grpo.html`
- **Dr. GRPO normalization critique** → `arXiv:2503.20783`

Concept-to-code map for the chapter

- **R1-Zero training regime (no cold start)** → `DeepSeek-R1/README.md`
- **rule-based accuracy/format rewards** → `DeepSeek-R1/DeepSeek_R1.pdf`
- **group-relative advantage, GRPO origin** → `arXiv:2402.03300 (DeepSeekMath)`
- **PPO clipped-ratio ancestor** → `arXiv:1707.06347`
- **token-mean loss,** `loss_agg_mode` → `verl.readthedocs.io /en/latest/algo/grpo.html`
- **Dr. GRPO normalization critique** → `arXiv:2503.20783`

This chapter's public sources are papers, a model card, and one docs page — not an executable DeepSeek trainer. The DeepSeek-R1 chapter (9) is where those same ideas meet real veRL configuration objects.

What carries forward

Sample a group. Score it with a rule. Compare within the group.
Move the policy, guarded — and audit what the rule never measured.

- The mechanism is now fully computable: reward, advantage, ratio, clip, KL placement, token mask, reduction.
- The limit is equally load-bearing: rule rewards see only what extraction and verification expose.

What carries forward

Sample a group. Score it with a rule. Compare within the group.
Move the policy, guarded — and audit what the rule never measured.

- The mechanism is now fully computable: reward, advantage, ratio, clip, KL placement, token mask, reduction.
- The limit is equally load-bearing: rule rewards see only what extraction and verification expose.

Next: Chapter 9 keeps this GRPO mechanism and adds cold-start data, rejection-sampled SFT, and staged RL to reach a usable, readable model family — DeepSeek-R1.